

Q1. Explain the difference between var, let, and const with examples.

Answer: In JavaScript, **var**, **let**, and **const** are used to declare variables. The main difference is their **scope, re-declaration, and re-assignment**.

1. var

- Introduced in older JavaScript.
- Has **function scope**.
- Can be **re-declared** and **re-assigned**.
- Its value can change anytime.

Example:

```
var name = "Rishabh";
var name = "Rahul";
name = "Amit";
console.log(name);
```

2. let

- Introduced in ES6.
- Has **block scope**.
- Cannot be re-declared in the same block.
- Can be re-assigned.

Example:

```
let age = 20;
age = 21;
console.log(age);
```

3. const

- Introduced in ES6.
- Has **block scope**.
- Cannot be re-declared or re-assigned.
- Used for fixed values.

Example:

```
const PI = 3.14;
console.log(PI);
```

Difference Table

Feature	var	let	const
Scope	Function	Block	Block
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No
Use	Old JavaScript	Changing values	Fixed values

Q2. How does JavaScript handle user events? Explain with examples.

Answer: A **JavaScript event** is an action performed by the user or browser, such as clicking a button, pressing a key, or moving the mouse. JavaScript responds to these actions using **event handlers**.

Common User Events

- ☐ **click** – Triggered when the user clicks an element once.
- ☐ **dblclick** – Triggered when the user double-clicks an element.
- ☐ **mouseover** – Triggered when the mouse pointer moves over an element.
- ☐ **mouseout** – Triggered when the mouse pointer leaves an element.
- ☐ **keydown** – Triggered when a keyboard key is pressed.
- ☐ **keyup** – Triggered when a pressed keyboard key is released.
- ☐ **submit** – Triggered when a form is submitted.

Example 1: Button Click

```
<button onclick="showMsg()">Click Me</button>
```

```
<script>
```

```
function showMsg(){  
    alert("Button Clicked!");  
}
```

```
</script>
```

When the user clicks the button, the message is displayed.

Example 2: Mouse Over

```
element.onmouseover = function(){  
    alert("Mouse is over the element");  
}
```

Advantages

- Makes web pages interactive.
- Improves user experience.
- Responds instantly to user actions.
- Creates dynamic websites.

Q3. What is the difference between == and === in JavaScript?

Answer: JavaScript provides two comparison operators: == (**Loose Equality**) and === (**Strict Equality**).

1. == (Loose Equality)

- Compares only values.
- Automatically converts data types if needed.

Example:

```
5 == "5"
```

Output: true

2. === (Strict Equality)

- Compares both value and data type.
- No type conversion.

Example:

5 === "5"

Output: false

More Examples

10 == "10" // true

10 === "10" // false

true == 1 // true

true === 1 // false

==

Compares value only

Type conversion occurs

Less secure

===

Compares value and type

No type conversion

More accurate

Difference Table

Q4. Explain JavaScript functions and their different types.

Answer: A **function** is a block of code that performs a specific task. Functions help reduce code repetition and improve readability.

Syntax

```
function functionName(){
    // code
}
```

Example

```
function greet(){
    console.log("Hello");
}
greet();
```

Types of Functions

1. Named Function

Has a function name.

```
function add(a,b){
    return a+b;
}
```

2. Anonymous Function

Has no name.

```
let msg = function(){
    console.log("Hello");
}
```

3. Arrow Function

Short syntax introduced in ES6.

```
const add = (a,b) => a+b;
```

4. Function with Parameters

```
function square(x){  
    return x*x;  
}
```

Advantages

- Reusable code
- Easy to maintain
- Reduces repetition
- Improves readability

Q5. What is the Document Object Model (DOM)? Explain its importance.

Answer: The **Document Object Model (DOM)** is a programming interface that represents an HTML document as a **tree of objects**. JavaScript uses the DOM to access and modify web page elements.

DOM Structure

```
Document  
|  
HTML  
/  \  
Head Body  
|  
Elements
```

Example

```
<p id="demo">Hello</p>  
<script>  
document.getElementById("demo").innerHTML = "Welcome";  
</script>
```

The text changes from **Hello** to **Welcome**.

Importance of DOM

- Access HTML elements.
- Change text and content.
- Modify CSS styles.
- Handle user events.
- Create dynamic web pages.

Common DOM Methods

- `getElementById()`

- `getElementsByClassName()`
- `querySelector()`
- `createElement()`
- `appendChild()`

Q6. Explain the Window Object and its commonly used methods.

Answer: The **Window Object** is the top-level object in JavaScript. It represents the browser window and provides methods to control the browser and interact with the user. All global variables and functions are part of the Window Object.

Commonly Used Methods

1. `alert()`

Displays a message in a popup box.

Example:

```
alert("Welcome to JavaScript");
```

2. `prompt()`

Takes input from the user.

Example:

```
let name = prompt("Enter your name");
```

3. `confirm()`

Displays a confirmation box with **OK** and **Cancel** buttons.

Example:

```
let result = confirm("Do you want to continue?");
```

4. `open()`

Opens a new browser window or tab.

Example:

```
window.open("https://www.google.com");
```

5. `close()`

Closes the current browser window.

Example:

```
window.close();
```

Advantages

- Displays messages to users.
- Accepts user input.
- Opens and closes browser windows.
- Controls browser operations.

Q7. What are JavaScript Objects? Explain with an example.

Answer: A **JavaScript Object** is a collection of **properties** and **methods**. It stores data in the form of **key-value pairs**. Objects are used to represent real-world entities like students, cars, employees, etc.

Syntax

```
let objectName = {  
    key: value  
};
```

Example

```
let student = {  
    name: "Rishabh",  
    age: 20,  
    course: "B.Sc. IT"  
};  
console.log(student.name);  
console.log(student.age);
```

Output:

```
Rishabh  
20
```

Accessing Object Properties

- Dot notation

```
student.name
```

- Bracket notation

```
student["age"]
```

Advantages

- Stores related data together.
- Makes code organized.
- Easy to access and update values.
- Used in almost every JavaScript application.

Q8. Explain the different looping statements available in JavaScript.

Answer: A **loop** is used to execute a block of code repeatedly until a condition becomes false. JavaScript provides different types of loops.

1. for Loop

Used when the number of iterations is known.

Example:

```
for(let i=1; i<=5; i++){
```

```
console.log(i);
```

```
}
```

2. while Loop

Executes while the condition is true.

Example:

```
let i=1;
```

```
while(i<=5){
```

```
    console.log(i);
```

```
    i++;
```

```
}
```

3. do...while Loop

Executes the code at least once.

Example:

```
let i=1;
```

```
do{
```

```
    console.log(i);
```

```
    i++;
```

```
}while(i<=5);
```

4. for...of Loop

Used to iterate through arrays and strings.

Example:

```
let colors=["Red","Blue","Green"];
```

```
for(let color of colors){
```

```
    console.log(color);
```

```
}
```

Advantages

- Reduces code repetition.
- Saves time.
- Makes programs shorter.
- Easy to process arrays and collections.

Q9. What is Event Bubbling and Event Capturing in JavaScript?

Answer: When an event occurs on an HTML element, JavaScript decides the order in which event handlers are executed. This process is called **Event Propagation**, which includes **Event Bubbling** and **Event Capturing**.

1. Event Bubbling

In **event bubbling**, the event starts from the **target element** and moves upward to its parent elements.

Example:

```
button.addEventListener("click", function(){
    alert("Button Clicked");
});
```

If a button is inside a <div>, clicking the button first triggers the button event, then the <div> event.

2. Event Capturing

In **event capturing**, the event starts from the **outer parent element** and moves downward to the target element.

Example:

```
div.addEventListener("click", function(){
    alert("Div Clicked");
}, true);
```

Difference

Event Bubbling	Event Capturing
Bottom to Top	Top to Bottom
Default behaviour	Enabled using true
Target executes first	Parent executes first

Q10. Explain JavaScript Type Conversion with suitable examples.

Answer: Type Conversion is the process of converting one data type into another. JavaScript supports **Implicit** and **Explicit** type conversion.

1. Implicit Type Conversion

JavaScript automatically converts the data type.

Example:

```
let result = "10" + 5;
console.log(result);
```

Output:

105

2. Explicit Type Conversion

The programmer manually converts the data type using functions.

String to Number

```
let num = Number("100");
console.log(num);
```

Number to String

```
let str = String(200);
console.log(str);
```

Boolean Conversion

```
Boolean(1); // true
Boolean(0); // false
```


Common Conversion Functions

- Number()
- String()
- Boolean()
- parseInt()
- parseFloat()

Advantages

- Converts data into the required format.
- Prevents type-related errors.
- Makes calculations and comparisons easier.

Q11. How does jQuery simplify DOM manipulation compared to JavaScript?

Answer: DOM Manipulation means changing HTML elements, text, attributes, or styles using code.

In plain JavaScript, we need longer code to access and modify elements. jQuery provides short and easy methods.

Plain JavaScript

```
document.getElementById("msg").innerHTML = "Hello";
```

jQuery

```
$("#msg").html("Hello");
```

Both codes change the text, but jQuery uses less code.

How jQuery simplifies DOM manipulation

- Shorter syntax
- Easy element selection
- Easy CSS changes
- Easy event handling
- Works across different browsers

Common jQuery methods

- html() – change HTML content
- text() – change text
- css() – change style
- attr() – change attributes
- append() – add new elements

Example

```
$("p").css("color", "blue");
```

This changes all paragraph text colour to blue.

Advantages

- Less coding
- Better readability
- Faster development
- Easier maintenance

Q12. Explain jQuery selectors with suitable examples.

Answer: jQuery selectors are used to find and select HTML elements. They work similarly to CSS selectors.

Syntax

`$(selector)`

Types of selectors

1. Element Selector: Selects all elements of a type.

```
$("p").hide();
```

Hides all paragraph elements.

2. ID Selector: Selects one element by ID.

```
$("#title").css("color", "red");
```

Changes the colour of the element with id title.

3. Class Selector: Selects all elements with a class.

```
$(".box").show();
```

Shows all elements having class box.

4. Universal Selector : Selects all elements.

```
$("*").hide();
```

Advantages

- Easy element selection
- Similar to CSS
- Reduces code length
- Improves readability

Q13. What are jQuery effects? Explain any four methods.

Answer: jQuery effects are built-in methods used to show, hide, fade, and animate HTML elements.

1. hide() : Hides an element.

```
$("#box").hide();
```

2. show() : Displays a hidden element.

```
$("#box").show();
```

3. fadeIn() : Gradually shows an element.

```
$("#box").fadeIn();
```

4. fadeOut() : Gradually hides an element.

```
$("#box").fadeOut();
```

Effect Table

Method	Purpose
hide()	Hide element
show()	Show element
fadeIn()	Gradually show
fadeOut()	Gradually hide

Advantages

- Attractive UI
- Smooth transitions
- Better user experience
- Easy to implement

Q14. How does jQuery AJAX simplify asynchronous communication?

Answer: AJAX (Asynchronous JavaScript and XML) allows a web page to send and receive data from a server without reloading the page.

jQuery provides simple methods for AJAX communication.

Basic AJAX syntax

```
$.ajax({  
  url: "data.php",  
  type: "GET",  
  success: function(result){  
    console.log(result);  
  }  
});
```

Important properties

- url – server file address
- type – GET or POST
- success – runs when data is received
- error – runs if request fails

Short methods

- \$.get() – fetch data
- \$.post() – send data
- \$.load() – load HTML into an element

Example

```
$.get("data.txt", function(data){  
    $("#result").html(data);  
});
```

This loads data from data.txt into the element with id result.

Advantages

- No page refresh
- Faster response
- Less code than plain JavaScript
- Better user experience

Q15. Explain jQuery Traversing methods with examples.

Answer: Traversing means moving through the DOM tree to find parent, child, or sibling elements.

1. parent() : Selects the immediate parent.

```
$("p").parent();
```

2. children() : Selects all direct child elements.

```
$("div").children();
```

3. find() : Searches for matching descendants.

```
$("div").find("p");
```

4. siblings() : Selects elements at the same level.

```
$("h1").siblings();
```

Example

```
<div>  
    <p>Paragraph</p>  
    <span>Text</span>  
</div>
```

- \$("p").parent() → div
- \$("div").children() → p and span
- \$("div").find("p") → p
- \$("p").siblings() → span

Advantages

- Easy DOM navigation

- Reduces complex code
- Quickly finds related elements
- Improves readability

Q16. Explain the advantages of HTML5 over previous HTML versions.

Answer: **HTML5** is the latest version of HTML used to create modern web pages. It provides many new features that make websites faster, more interactive, and easier to develop compared to older HTML versions.

Advantages of HTML5

- 1. Semantic Elements:** HTML5 introduces semantic tags like `<header>`, `<nav>`, `<section>`, and `<footer>`, making code more meaningful and easier to understand.
- 2. Multimedia Support:** HTML5 allows audio and video to be added directly using `<audio>` and `<video>` tags without external plugins like Flash.
- 3. Better Forms:** HTML5 provides new input types such as email, date, number, and password, making form validation easier.
- 4. Improved Graphics:** It supports `<canvas>` and SVG for creating graphics, animations, and games.
- 5. Cross-Platform Support:** HTML5 works on computers, tablets, and mobile devices with all modern browsers.
- 6. Better Performance:** HTML5 websites load faster and provide a better user experience.

Advantages Summary

- Simple and clean code
- Multimedia support
- Better forms
- Mobile-friendly
- Faster performance
- Improved SEO using semantic tags

Q17. What are Semantic Elements in HTML5? Explain any five semantic tags.

Answer: **Semantic elements** are HTML5 tags that clearly describe the **meaning and purpose** of the content they contain. They make the webpage easier to understand for developers, browsers, and search engines.

Common Semantic Tags

- 1. `<header>`:** Defines the **top section** of a webpage or article. It usually contains the logo, title, or navigation.

Example:

```
<header>
```

```
  <h1>My Website</h1>
```

```
</header>
```

- 2. `<nav>`:** Defines the **navigation menu** containing links to different pages.

Example:

```
<nav>

  <a href="home.html">Home</a>

  <a href="about.html">About</a>

</nav>
```

3. <section>: Represents a **separate section** of related content on a webpage.

Example:

```
<section>

  <h2>About Us</h2>

  <p>Welcome to our website.</p>

</section>
```

4. <article>: Represents an **independent piece of content**, such as a blog post, news article, or magazine article.

Example:

```
<article>

  <h2>Latest News</h2>

  <p>New product launched today.</p>

</article>
```

5. <footer>: Defines the **bottom section** of a webpage. It usually contains copyright information or contact details.

Example:

```
<footer>

  <p>Copyright © 2026</p>

</footer>
```

Advantages of Semantic Elements

- Improves code readability.
- Makes webpages SEO-friendly.
- Improves accessibility.
- Easy to maintain.
- Clearly defines the structure of a webpage.

Q18. Differentiate between <div> and semantic HTML5 elements with examples.

Answer: The **<div>** element is a general-purpose container that has **no specific meaning**, while **semantic HTML5 elements** describe the purpose of the content they contain.

Difference Table

<div>	Semantic Elements
No meaning	Meaningful tags
Used as a generic container	Used for specific sections
Less SEO-friendly	Better for SEO
Harder to understand	Easy to understand
Example: <div>	Examples: <header>, <nav>, <section>, <footer>

Example using <div>

```
<div>
  <h1>My Website</h1>
</div>
```

The <div> tag only groups content and does not describe its purpose.

Example using Semantic Elements

```
<header>
  <h1>My Website</h1>
</header>
<nav>
  <a href="#">Home</a>
</nav>
<footer>
  Copyright © 2026
</footer>
```

Here, each tag clearly describes its purpose.

Advantages of Semantic Elements

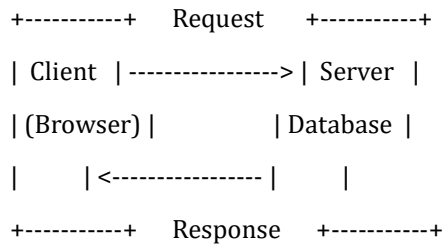
- Better code readability.
- Improves SEO.
- Easy to maintain.
- Better accessibility.
- Helps browsers and search engines understand the webpage.

Q19. Explain the Client-Server Architecture with a neat diagram.

Answer: Client-Server Architecture is a model in which the **client** sends a request to the **server**, and the server processes the request and sends back a response.

- **Client:** A user's device or web browser that requests information.
- **Server:** A powerful computer that stores data, processes requests, and provides services.

Neat Diagram



Working

1. User enters a website URL.
2. The browser (client) sends a request to the server.
3. The server processes the request.
4. The server sends the required webpage back to the client.
5. The browser displays the webpage.

Advantages

- Easy data sharing.
- Better security.
- Centralized data storage.
- Easy maintenance.
- Supports multiple users.

Example

When you open www.google.com, your browser sends a request to Google's server, and the server returns the search page.

Q20. Differentiate between Client-Side Scripting and Server-Side Scripting with examples.

Answer: Scripting is used to make websites dynamic. It is of two types: **Client-Side Scripting** and **Server-Side Scripting**.

Difference Table

Client-Side Scripting	Server-Side Scripting
Runs in the browser	Runs on the server
Faster execution	Slightly slower
Used for UI and validation	Used for database and business logic
Code is visible to users	Code is hidden from users
Example: JavaScript	Example: PHP, Node.js, Python

Client-Side Example

```

function showMessage() {
    alert("Welcome!");
}
  
```


Server-Side Example (PHP)

```
<?php  
echo "Welcome";  
?>
```

Advantages of Client-Side

- Fast execution
- Reduces server load
- Better user experience

Advantages of Server-Side

- More secure
- Handles databases
- Processes business logic

Q21. Explain the purpose of the `addEventListener()` method with an example. How does it differ from inline events?

Answer: The `addEventListener()` method is used to attach an event to an HTML element without writing JavaScript inside the HTML tag.

Syntax

```
element.addEventListener("event", function);
```

Example

```
<button id="btn">Click Me</button>  
  
<script>  
document.getElementById("btn").addEventListener("click", function(){  
    alert("Button Clicked");  
});  
</script>
```

When the button is clicked, an alert message is displayed.

Difference between `addEventListener()` and Inline Events

<code>addEventListener()</code>	Inline Event
JavaScript is written separately	JavaScript is written inside HTML
Cleaner code	Mixed HTML and JavaScript
Multiple events can be added	Usually one event only
Easy to maintain	Harder to maintain

Advantages

- Keeps HTML clean.

- Easy to maintain.
- Supports multiple event listeners.
- Reusable code.

Q22. What is Event Delegation in JavaScript? Why is it useful?

Answer: Event Delegation is a technique where a **single event listener** is attached to a parent element instead of adding separate event listeners to each child element.

The parent handles events that occur on its child elements using **event bubbling**.

Example

```
<ul id="list">
  <li>Apple</li>
  <li>Mango</li>
  <li>Banana</li>
</ul>
<script>
document.getElementById("list").addEventListener("click", function(event){
  alert(event.target.innerHTML);
});
</script>
```

When any list item is clicked, its name is displayed.

Advantages

- Reduces the number of event listeners.
- Improves performance.
- Saves memory.
- Works for dynamically added elements.
- Makes code easier to manage.

Q23. Explain different String methods in JavaScript with examples.

Answer: String methods are built-in JavaScript functions used to perform operations on text.

1. length

Returns the number of characters.

```
let str = "JavaScript";
console.log(str.length);
```

2. toUpperCase()

Converts text to uppercase.

```
console.log(str.toUpperCase());
```

3. toLowerCase()

Converts text to lowercase.

```
console.log(str.toLowerCase());
```

4. substring()

Returns part of a string.

```
console.log(str.substring(0,4));
```

5. replace()

Replaces one word with another.

```
let text = "Hello World";
```

```
console.log(text.replace("World","JavaScript"));
```

Advantages

- Easy text manipulation.
- Improves string handling.
- Saves coding time.
- Makes programs efficient.

Q24. Explain different Math object methods in JavaScript.

Answer: The **Math Object** provides built-in methods to perform mathematical calculations. It does not need to be created using new.

Common Math Methods

1. Math.round()

Rounds a number to the nearest integer.

```
Math.round(4.6); // 5
```

2. Math.floor()

Returns the largest integer less than or equal to the number.

```
Math.floor(4.9); // 4
```

3. Math.ceil()

Returns the smallest integer greater than or equal to the number.

```
Math.ceil(4.2); // 5
```

4. Math.sqrt()

Returns the square root of a number.

```
Math.sqrt(25); // 5
```

5. Math.random()

Generates a random number between **0** and **1**.

```
Math.random();
```

Advantages

- Performs mathematical calculations easily.

- Saves programming time.
- Useful for games, calculations, and random number generation.
- Provides many built-in mathematical functions.

Q25. Explain Date Object methods in JavaScript.

Answer: The **Date Object** in JavaScript is used to **create, display, and manipulate date and time**. It helps programmers work with the current date, current time, day, month, and year.

Syntax

```
let date = new Date();
```

Common Date Object Methods

1. getDate()

Returns the day of the month.

Example:

```
let d = new Date();  
console.log(d.getDate());
```

2. getMonth()

Returns the current month (0 = January, 11 = December).

Example:

```
console.log(d.getMonth());
```

3. getFullYear()

Returns the current year.

Example:

```
console.log(d.getFullYear());
```

4. getHours()

Returns the current hour.

Example:

```
console.log(d.getHours());
```

5. getMinutes()

Returns the current minutes.

Example:

```
console.log(d.getMinutes());
```

6. getSeconds()

Returns the current seconds.

Example:

```
console.log(d.getSeconds());
```

Advantages

- Displays current date and time.
- Performs date and time calculations.
- Useful for calendars, clocks, and scheduling.
- Easy to format and manage dates.

Q26. What is Browser Object Model (BOM)? Explain its commonly used objects.

Answer: The **Browser Object Model (BOM)** is a collection of objects provided by the browser that allows JavaScript to interact with the browser window. BOM helps control browser features such as windows, history, location, and screen information.

Commonly Used BOM Objects

1. Window Object

It is the main object of the BOM and represents the browser window.

Example:

```
alert("Welcome!");
```

2. Navigator Object

Provides information about the browser.

Example:

```
console.log(navigator.userAgent);
```

3. Location Object

Provides information about the current webpage URL.

Example:

```
console.log(location.href);
```

4. History Object

Allows moving backward and forward in browser history.

Example:

```
history.back();
```

```
history.forward();
```

5. Screen Object

Provides information about the user's screen.

Example:

```
console.log(screen.width);
```

```
console.log(screen.height);
```

Advantages

- Controls browser windows.
- Accesses browser information.
- Manages webpage navigation.

- Improves user interaction.

Q27. Explain JavaScript Error Handling using try, catch, throw, and finally.

Answer: Error Handling is used to handle runtime errors so that the program does not stop unexpectedly. JavaScript provides **try, catch, throw, and finally** for this purpose.

1. **try:** The try block contains code that may generate an error.
2. **catch:** The catch block handles the error if one occurs.
3. **throw:** The throw statement is used to create a custom error.
4. **finally:** The finally block always executes, whether an error occurs or not.

Example

```
try {  
    let num = 10;  
    if (num < 20) {  
        throw "Number is too small";  
    }  
}  
catch(error) {  
    console.log(error);  
}  
finally {  
    console.log("Program Finished");  
}
```

Advantages

- Prevents program crashes.
- Handles unexpected errors.
- Makes debugging easier.
- Improves program reliability.

Q28. What is Strict Mode in JavaScript? Explain its advantages.

Answer: Strict Mode is a special mode in JavaScript that helps write safer and error-free code. It prevents the use of incorrect or unsafe coding practices.

To enable Strict Mode, write "use strict"; at the beginning of the script or function.

Syntax

```
"use strict";
```

Example

```
"use strict";
```

```
x = 10; // Error
```

In Strict Mode, variables must be declared using let, const, or var.

Correct code:

```
"use strict";
```

```
let x = 10;
```

```
console.log(x);
```

Advantages

- Prevents accidental creation of variables.
- Helps detect coding mistakes.
- Improves code security.
- Makes debugging easier.
- Encourages writing clean and reliable code.

Q29. Explain the advantages of using jQuery over plain JavaScript.

Answer: jQuery is a lightweight JavaScript library that makes web development easier. Compared to plain JavaScript, jQuery uses shorter syntax and provides many built-in functions for common tasks.

Advantages of jQuery

1. Shorter and Simpler Code: jQuery requires less code than plain JavaScript.

JavaScript:

```
document.getElementById("demo").innerHTML = "Hello";
```

jQuery:

```
$("#demo").html("Hello");
```

2. Easy DOM Manipulation: It provides simple methods to change HTML content, CSS, and attributes.

3. Easy Event Handling: Events like click(), mouseover(), and submit() can be handled with simple methods.

4. Built-in Effects and Animations: jQuery provides methods like hide(), show(), fadeIn(), fadeOut(), and slideToggle() without writing complex code.

5. AJAX Support: AJAX methods like \$.get() and \$.post() make server communication easy without refreshing the page.

6. Cross-Browser Compatibility: The same jQuery code works on all major browsers without extra changes.

Advantages Summary

- Less code
- Easy to learn
- Fast development
- Cross-browser support
- Built-in animations and AJAX

- Easy DOM manipulation

Q30. Explain the role of HTML5 APIs in modern web development.

Answer: **HTML5 APIs** are built-in programming interfaces that allow web applications to perform advanced tasks without using external plugins. They make modern websites faster, more interactive, and user-friendly.

Common HTML5 APIs

1. Geolocation API

Gets the user's current location.

Example:

```
navigator.geolocation.getCurrentPosition(showPosition);
```

2. Web Storage API

Stores data in the browser using **localStorage** and **sessionStorage**.

Example:

```
localStorage.setItem("name", "Rishabh");
```

3. Canvas API

Used to draw graphics, shapes, animations, and games.

4. Drag and Drop API

Allows users to drag and drop elements on a webpage.

5. Audio and Video API

Plays audio and video without using external plugins.

Example:

```
<video controls>  
  <source src="video.mp4">  
</video>
```

Advantages of HTML5 APIs

- Makes websites more interactive.
- Improves user experience.
- Reduces dependency on plugins.
- Supports multimedia and graphics.
- Works on desktop and mobile browsers.